

NumPy I — Arrays, Indexing, Slicing & Broadcasting

By the end of today you will understand why NumPy exists, how to create and work with arrays, how to slice and index them, and how broadcasting saves you from writing loops. NumPy is the foundation that everything in data science is built on — Pandas, scikit-learn, TensorFlow, all of it uses NumPy underneath.

SECTION 01 Install NumPy Now — Do This Before Reading Further

NumPy is not part of Python's standard library. You need to install it once using pip. Since you have Python 3 and VS Code set up, this takes about 30 seconds.

✂ Step-by-step installation

1. Open VS Code.
2. Open the integrated terminal: Terminal menu → New Terminal (or press Ctrl + ` on Windows/Linux, Cmd + ` on Mac).
3. In the terminal, type exactly this and press Enter:

```
pip install numpy      ← Windows
pip3 install numpy     ← Mac or Linux
```

4. Wait for it to finish. You will see a message like: Successfully installed numpy-1.x.x
5. Verify it worked. In the terminal type:

```
python -c "import numpy; print(numpy.__version__)"
```

You should see a version number like 1.26.4 or 2.0.0
If you see a version number — NumPy is installed and ready.

You only install NumPy once. After that, you just write `import numpy as np` at the top of any file and it is ready to use. The alias `np` is the universal standard — every data scientist in the world uses it. Never use a different alias.

SECTION 02 What is NumPy and Why Does It Exist?

You already know Python lists. They store collections of items. So why do we need something new? The answer is speed and math. Lots of it.

The problem with Python lists for data science

Imagine you have 1 million customer purchase amounts and you want to add 18% GST to every single one. With a Python list, you would write a loop:

```
prices      = [199, 349, 99, 599, ...]    # 1 million items
new_prices  = []

for price in prices:
    new_prices.append(price * 1.18)        # This loop runs 1 MILLION times
```

This works. But it is slow. On a modern laptop this loop takes several seconds for a million items. In data science, you work with millions — sometimes billions — of data points. Loops are simply not fast enough.

NumPy solves this by doing the entire operation in one shot, without a loop, using optimised C code running behind the scenes. The same operation on 1 million items takes milliseconds instead of seconds.

```
import numpy as np

prices      = np.array([199, 349, 99, 599])    # NumPy array
new_prices  = prices * 1.18                    # No loop needed!
print(new_prices)
```

Output:

```
[234.82 411.82 116.82 707.82]
```

Key differences: Python list vs NumPy array

Python list — can hold mixed types (int, string, bool together). Slow for math.

NumPy array — holds ONE type only (all ints or all floats). Extremely fast for math.

Think of a Python list as a general-purpose bag that holds anything.

Think of a NumPy array as a specialised container designed purely for numbers and speed.

There are many ways to create a NumPy array. Each one is designed for a specific situation. Learn all eight — you will use every one of them during this program.

Method 1 — From a Python list

The most common way. Convert an existing list into a NumPy array.

```
import numpy as np

# 1D array (like a single row of data)
scores = np.array([85, 92, 78, 95, 88])
print(scores)
print(type(scores))      # <class 'numpy.ndarray'>
print(scores.dtype)      # int64 (the data type of elements)

# 2D array (like a table with rows and columns)
marks_table = np.array([
    [85, 92, 78],      # Student 1: Maths, Science, English
    [90, 85, 88],      # Student 2
    [70, 75, 80]       # Student 3
])
print(marks_table)
print(marks_table.shape) # (3, 3) — 3 rows, 3 columns
```

Output:

```
[85 92 78 95 88]
<class 'numpy.ndarray'>
int64
[[85 92 78]
 [90 85 88]
 [70 75 80]]
(3, 3)
```

Method 2 — np.zeros() and np.ones()

Create arrays filled with all zeros or all ones. Used constantly in ML to initialise model weights and create placeholders.

```
# Array of zeros
empty_scores = np.zeros(5)
print(empty_scores)      # [0. 0. 0. 0. 0.]

# 2D array of zeros (3 rows, 4 columns)
weight_matrix = np.zeros((3, 4))
print(weight_matrix)
```

```
# Array of ones
all_ones = np.ones((2, 3))
print(all_ones)
```

Output:

```
[0. 0. 0. 0. 0.]
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

Method 3 — np.arange()

Like Python's `range()` but returns a NumPy array instead of a sequence. You can use decimal steps too, which `range()` cannot do.

```
# Integers from 0 to 9
a = np.arange(10)
print(a)           # [0 1 2 3 4 5 6 7 8 9]

# From 1 to 20, step of 2
b = np.arange(1, 21, 2)
print(b)           # [ 1  3  5  7  9 11 13 15 17 19]

# Float steps — impossible with range(), easy with arange
c = np.arange(0.0, 1.1, 0.1)
print(c)           # [0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1. ]
```

Output:

```
[0 1 2 3 4 5 6 7 8 9]
[ 1  3  5  7  9 11 13 15 17 19]
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1. ]
```

Method 4 — np.linspace()

Generates a specific number of evenly spaced values between a start and end point. Unlike `arange()` where you specify the step, `linspace()` lets you specify how many points you want.

```
# 5 evenly spaced points between 0 and 1
a = np.linspace(0, 1, 5)
print(a)           # [0.  0.25 0.5  0.75 1. ]

# 10 evenly spaced temperatures between 0 and 100
temps = np.linspace(0, 100, 10)
print(temps)
```

Output:

```
[0.    0.25 0.5   0.75 1.   ]
[ 0.   11.1  22.2  33.3  44.4  55.6  66.7  77.8  88.9 100. ]
```

`linspace` is used constantly in ML for generating axis values for plots, learning rate schedules, and creating smooth curves for visualisation. You will see it almost every day from Phase 2 onwards.

Method 5 — `np.random` arrays

Generate arrays filled with random numbers. Absolutely essential in ML for creating test data, initialising model weights, and splitting datasets.

```
np.random.seed(42)      # Set seed so results are reproducible

# Random floats between 0.0 and 1.0
a = np.random.random((3, 3))
print(a)

# Random integers between 1 and 100
b = np.random.randint(1, 101, size=(4,))
print(b)

# Random values from a normal distribution
# (bell curve, mean=0, std=1)
c = np.random.randn(5)
print(c)
```

Output:

```
[[0.374 0.951 0.732]
 [0.599 0.156 0.058]
 [0.866 0.708 0.021]]
[52 87 12 74]
[-0.234  1.574 -0.463  0.242  1.913]
```

`np.random.seed(42)` sets a fixed starting point for randomness. This means every time you run the code you get the SAME random numbers. This is crucial in ML so your experiments are reproducible and comparable. Use it always.

Methods 6, 7, 8 — Special arrays

```
# np.full() — fill with any specific value
a = np.full((2, 4), fill_value=7)
print(a)      # [[7 7 7 7] [7 7 7 7]]
```

```
# np.eye() - identity matrix (1s on diagonal, 0s elsewhere)
# Used constantly in linear algebra and ML
b = np.eye(4)
print(b)

# np.empty() - uninitialized array (just reserves memory, faster)
# Values are garbage - you will fill them later
c = np.empty((2, 3))
print(c)
```

Output:

```
[[7 7 7 7]
 [7 7 7 7]]
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

SECTION 04 Understanding Array Shape, Size, and Dimensions

Before you can work with arrays confidently, you need to understand three key properties. These will appear in every single ML program you write.

Property	Code	What it tells you
shape	arr.shape	The dimensions. (3,) means 1D with 3 items. (3,4) means 3 rows, 4 columns.
ndim	arr.ndim	Number of dimensions. 1 for 1D, 2 for 2D (table), 3 for 3D (image).
size	arr.size	Total number of elements. A (3,4) array has size 12.
dtype	arr.dtype	Data type of elements: int64, float64, bool, etc.

```
import numpy as np

# A 2D array representing an exam results table
# Rows = students, Columns = subjects (Maths, Science, English, Hindi)
results = np.array([
    [85, 90, 78, 92],
    [72, 88, 95, 80],
    [91, 76, 83, 87]
])

print("Shape :", results.shape)    # (3, 4) - 3 students, 4 subjects
print("Dimensions:", results.ndim) # 2    - it is a 2D table
print("Total marks:", results.size)# 12   - 12 numbers in total
```

```
print("Data type:", results.dtype) # int64
```

Output:

```
Shape : (3, 4)
Dimensions: 2
Total marks: 12
Data type: int64
```

Understanding shape is CRITICAL in ML. When you build a neural network, every error message about "dimension mismatch" or "cannot broadcast" is a shape problem.

Rule of thumb:

- (n,) — 1D array. Like a single row of numbers.
- (m, n) — 2D array. Like a spreadsheet table.
- (m, n, k) — 3D array. Like a colour image (height x width x colour channels).

Reshaping arrays — changing shape without changing data

`reshape()` changes the shape of an array without changing its data or total number of elements. This is one of the most used operations in ML.

```
import numpy as np

# Start with a 1D array of 12 numbers
flat = np.arange(1, 13)
print(flat)          # [ 1  2  3  4  5  6  7  8  9 10 11 12]
print(flat.shape)    # (12,)

# Reshape into a 3x4 table (3 rows, 4 columns)
table = flat.reshape(3, 4)
print(table)
print(table.shape)   # (3, 4)

# Reshape into a 2x6 table
table2 = flat.reshape(2, 6)
print(table2)

# Use -1 to let NumPy calculate one dimension automatically
auto = flat.reshape(4, -1)    # 4 rows, NumPy figures out columns = 3
print(auto.shape)           # (4, 3)
```

Output:

```
[ 1  2  3  4  5  6  7  8  9 10 11 12]
(12,)
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

```
(3, 4)
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]]
(4, 3)
```

SECTION 05 Indexing and Slicing — Accessing Array Elements

Indexing and slicing in NumPy work similarly to Python lists but are much more powerful. In data science you constantly need to extract specific rows, columns, or regions of your data.

1D array indexing and slicing

Works exactly like a Python list. Index starts at 0. Negative indexing works from the end.

```
import numpy as np

scores = np.array([85, 92, 78, 95, 88, 70, 99])
# Index:          0    1    2    3    4    5    6

# Single element
print(scores[0])      # 85    (first)
print(scores[-1])     # 99    (last)
print(scores[3])      # 95

# Slicing [start : end : step]
print(scores[1:4])    # [92 78 95] (index 1 to 3)
print(scores[:3])     # [85 92 78] (first 3)
print(scores[4:])     # [88 70 99] (from index 4 to end)
print(scores[::-2])   # [85 78 88 99] (every other element)
print(scores[::-1])   # [99 70 88 95 78 92 85] (reversed!)
```

Output:

```
85
99
95
[92 78 95]
[85 92 78]
[88 70 99]
[85 78 88 99]
[99 70 88 95 78 92 85]
```

2D array indexing — rows and columns

For a 2D array, you specify two indices: [row, column]. This is the most important concept in this section for data science work.

```

import numpy as np

# Exam results: 4 students x 3 subjects (Maths, Science, English)
results = np.array([
    [85, 90, 78], # Student 0: Rahul
    [72, 88, 95], # Student 1: Priya
    [91, 76, 83], # Student 2: Arjun
    [60, 70, 65]  # Student 3: Sneha
])

# Single element: [row, column]
print(results[0, 0]) # 85 (Rahul's Maths mark)
print(results[1, 2]) # 95 (Priya's English mark)
print(results[-1, -1]) # 65 (Sneha's English mark)

# Entire row (one student's results)
print(results[1]) # [72 88 95] (Priya's all marks)
print(results[1, :]) # [72 88 95] (same, explicit)

# Entire column (all students' marks for one subject)
print(results[:, 0]) # [85 72 91 60] (all Maths marks)
print(results[:, 1]) # [90 88 76 70] (all Science marks)

# Sub-table: first 2 students, last 2 subjects
print(results[:2, 1:])

```

Output:

```

85
95
65
[72 88 95]
[72 88 95]
[85 72 91 60]
[90 88 76 70]
[[90 78]
 [88 95]]

```

The colon : means 'everything'. It is the most important symbol in NumPy slicing.

```

results[:, 0] — all rows, column 0 → all students' Maths score
results[0, :] — row 0, all columns → student 0's all subject scores
results[:, :] — everything → the whole table

```

Boolean (fancy) indexing — filter with a condition

This is one of the most powerful features in NumPy and is used constantly in data science. Instead of an index number, you pass a True/False condition and NumPy returns only the elements where the condition is True.

```

import numpy as np

scores = np.array([85, 92, 78, 95, 38, 70, 99, 45, 30])

# Which students passed? (marks >= 40)
passed_mask = scores >= 40
print(passed_mask)
# [ True  True  True  True False  True  True  True False]

# Get only the passing scores
passed_scores = scores[passed_mask]
print(passed_scores)    # [85 92 78 95 70 99 45]

# Or do it in one line (most common way)
print(scores[scores >= 40])    # [85 92 78 95 70 99 45]
print(scores[scores > 90])     # [92 95 99]

# Count how many passed
print(np.sum(scores >= 40))    # 7

```

Output:

```

[ True  True  True  True False  True  True  True False]
[85 92 78 95 70 99 45]
[85 92 78 95 70 99 45]
[92 95 99]
7

```

Boolean indexing is how you filter data in NumPy. In Pandas (Day 7), the exact same concept is used to filter rows in a DataFrame. Master it here and it will feel familiar when you get there.

SECTION 06 Array Operations — Math Without Loops

One of NumPy's biggest strengths is that mathematical operations apply to the ENTIRE array at once, without writing a single loop. This is called vectorisation.

Element-wise arithmetic

When you add, subtract, multiply, or divide two arrays of the same shape, NumPy does the operation element by element automatically.

```

import numpy as np

maths    = np.array([85, 72, 91, 60])
science  = np.array([90, 88, 76, 70])

```

```

# Add arrays element by element
total = maths + science
print(total)          # [175 160 167 130]

# Subtract
diff = maths - science
print(diff)           # [ -5 -16  15 -10]

# Multiply each element by a scalar (single number)
doubled = maths * 2
print(doubled)        # [170 144 182 120]

# Calculate average of both subjects for each student
avg = (maths + science) / 2
print(avg)            # [87.5 80.  83.5 65. ]

# Apply GST to prices – entire array in one line
prices = np.array([100, 250, 80, 500, 120])
with_gst = prices * 1.18
print(with_gst)       # [118.  295.  94.4 590. 141.6]

```

Output:

```

[175 160 167 130]
[ -5 -16  15 -10]
[170 144 182 120]
[87.5 80.  83.5 65. ]
[118.  295.  94.4 590. 141.6]

```

Universal Functions (ufuncs) — math on every element

NumPy provides a large library of mathematical functions that work on entire arrays at once. These are called universal functions or ufuncs.

```

import numpy as np

data = np.array([1, 4, 9, 16, 25, 36])

print(np.sqrt(data))      # [1.  2.  3.  4.  5.  6.]
print(np.square(data))    # [  1  16  81 256 625 1296]
print(np.log(data))       # natural log of each element
print(np.exp(data))       # e raised to power of each element
print(np.abs([-3, 5, -7, 2])) # [3 5 7 2] (absolute value)

```

Output:

```

[1.  2.  3.  4.  5.  6.]
[  1  16  81 256 625 1296]
[0.      1.386 2.197 2.773 3.219 3.584]
[ 2.718 54.598 8103.08 ...]
[3 5 7 2]

```

Aggregate functions — summary statistics

These functions collapse an entire array (or rows/columns) into a single summary value. You will use these every day in data science.

```
import numpy as np

scores = np.array([85, 92, 78, 95, 88, 70, 99, 63])

print("Sum      :", np.sum(scores))      # 670
print("Mean     :", np.mean(scores))     # 83.75
print("Median   :", np.median(scores))   # 86.5
print("Max      :", np.max(scores))      # 99
print("Min      :", np.min(scores))      # 63
print("Std Dev  :", np.std(scores))      # 11.67 (standard deviation)
print("Variance:", np.var(scores))      # 136.19
```

Output:

```
Sum      : 670
Mean     : 83.75
Median   : 86.5
Max      : 99
Min      : 63
Std Dev  : 11.67
Variance: 136.19
```

Using axis to compute row-wise or column-wise

For 2D arrays, you can compute aggregates along a specific direction. `axis=0` means go down the rows (compute per column). `axis=1` means go across the columns (compute per row).

```
import numpy as np

# 3 students, 3 subjects
marks = np.array([
    [85, 90, 78],    # Rahul
    [72, 88, 95],    # Priya
    [91, 76, 83]     # Arjun
])

# Average per SUBJECT (across all students) — axis=0 (go down rows)
subject_avg = np.mean(marks, axis=0)
print("Subject averages:", subject_avg)    # [82.67 84.67 85.33]

# Average per STUDENT (across all subjects) — axis=1 (go across columns)
student_avg = np.mean(marks, axis=1)
print("Student averages:", student_avg)    # [84.33 85.   83.33]

# Total marks per student
```

```
student_total = np.sum(marks, axis=1)
print("Student totals :", student_total) # [253 255 250]
```

Output:

```
Subject averages: [82.67 84.67 85.33]
Student averages: [84.33 85.    83.33]
Student totals  : [253 255 250]
```

Remember the axis direction:

axis=0 — collapses ROWS → gives one result per COLUMN (vertical direction, going DOWN)

axis=1 — collapses COLUMNS → gives one result per ROW (horizontal direction, going ACROSS)

Memory trick: axis=0 goes the way rows are stacked (0 = vertical). axis=1 goes the way columns are lined up (1 = horizontal).

SECTION 07 Broadcasting — The Magic That Makes NumPy Powerful

Broadcasting

What it is: NumPy's ability to perform operations between arrays of different shapes without copying data. NumPy automatically 'stretches' the smaller array to match the shape of the larger one.

Real-life analogy: Imagine a restaurant bill. The table has 4 people and each person ordered a different dish. At the end you add a 10% service charge to everyone's bill. You do not calculate 4 separate service charges manually. You apply ONE rate to EVERYONE. Broadcasting is Python doing exactly this — applying one value or rule to an entire array.

In Python: `prices * 1.1` — the scalar 1.1 broadcasts across all elements

Scalar broadcasting — one value applied to all

When you do any operation between an array and a single number (scalar), the number is automatically applied to every element. You have already been doing this without knowing it had a name.

```
import numpy as np

prices = np.array([100, 250, 80, 500, 120])

# Add Rs. 50 shipping to every item
with_shipping = prices + 50
```

```

print(with_shipping)      # [150 300 130 550 170]

# Apply 20% discount to every item
discounted = prices * 0.80
print(discounted)         # [ 80. 200.  64. 400.  96.]

# Normalise all prices to 0-1 range
normalised = (prices - prices.min()) / (prices.max() - prices.min())
print(normalised)         # [0.048 0.4   0.    1.    0.095]

```

Output:

```

[150 300 130 550 170]
[ 80. 200.  64. 400.  96.]
[0.048 0.4   0.    1.    0.095]

```

That last line — normalisation — is one of the most important preprocessing steps in machine learning. It scales all values to be between 0 and 1 so no single feature dominates the model. You will use this exact formula in Phase 2.

Array-to-array broadcasting — different shapes working together

Broadcasting also works between two arrays of different shapes, as long as they are compatible. NumPy stretches the smaller array to match.

```

import numpy as np

# 4 students, each with 3 subject marks
marks = np.array([
    [85, 90, 78],
    [72, 88, 95],
    [91, 76, 83],
    [60, 70, 65]
])

# Bonus marks per subject (Maths +5, Science +3, English +2)
bonus = np.array([5, 3, 2])

# Add bonus to ALL students at once — broadcasting!
# bonus shape (3,) broadcasts against marks shape (4,3)
new_marks = marks + bonus
print(new_marks)

```

Output:

```

[[90 93 80]
 [77 91 97]
 [96 79 85]
 [65 73 67]]

```

NumPy automatically added [5, 3, 2] to every row of marks without you writing a loop. This is broadcasting. If you had done this with Python lists, you would need a nested for loop. With NumPy, it is one line.

SECTION 08 Practical Example — Complete Student Analysis

Let us put everything from today together into one complete, realistic program. This is the kind of analysis you will do constantly in data science.

```
import numpy as np

# — Dataset: 5 students, 4 subjects —————
students = ["Rahul", "Priya", "Arjun", "Sneha", "Mohammed"]
subjects = ["Maths", "Science", "English", "Hindi"]

marks = np.array([
    [85, 90, 78, 92],
    [72, 88, 95, 80],
    [91, 76, 83, 87],
    [60, 70, 65, 55],
    [95, 92, 89, 97]
])

print(f"Data shape: {marks.shape}")    # (5, 4)

# — Per-student analysis (axis=1) —————
totals = np.sum(marks, axis=1)
averages = np.mean(marks, axis=1)
print("\nStudent Results:")
for i, name in enumerate(students):
    grade = "PASS" if averages[i] >= 40 else "FAIL"
    print(f" {name:10}: Total={totals[i]}, Avg={averages[i]:.1f} — {grade}")

# — Per-subject analysis (axis=0) —————
subject_avg = np.mean(marks, axis=0)
print("\nSubject Averages:")
for i, sub in enumerate(subjects):
    print(f" {sub:10}: {subject_avg[i]:.1f}")

# — Overall class stats —————
print(f"\nClass highest : {np.max(marks)}")
print(f"Class lowest  : {np.min(marks)}")
print(f"Class average : {np.mean(marks):.1f}")

# — Who scored above 85 in any subject? —————
high_scorers = marks > 85
print(f"\nMarks above 85: {np.sum(high_scorers)} out of {marks.size}")
```

```
# — Normalise marks to 0-1 scale —————
normalised = (marks - marks.min()) / (marks.max() - marks.min())
print(f"\nNormalised (sample): {normalised[0]}")
```

Output:

```
Data shape: (5, 4)
```

Student Results:

```
Rahul      : Total=345, Avg=86.2 — PASS
Priya      : Total=335, Avg=83.8 — PASS
Arjun      : Total=337, Avg=84.2 — PASS
Sneha      : Total=250, Avg=62.5 — PASS
Mohammed   : Total=373, Avg=93.2 — PASS
```

Subject Averages:

```
Maths      : 80.6
Science    : 83.2
English    : 82.0
Hindi      : 82.2
```

```
Class highest : 97
Class lowest  : 55
Class average : 82.0
```

```
Marks above 85: 11 out of 20
```

```
Normalised (sample): [0.714 0.833 0.548 0.881]
```

SECTION 09 Interview Prep — Questions to Answer After Today

These questions come up in almost every data science and ML interview. Speak the answers out loud.

Q — What is NumPy and why is it better than a Python list for data science?

NumPy is a Python library for numerical computing. It provides the ndarray object — a fast, memory-efficient array. The key advantages over Python lists are: (1) Speed — NumPy operations are implemented in C and run 10-100 times faster than equivalent Python loops. (2) Vectorisation — mathematical operations apply to the entire array at once without writing loops. (3) Broadcasting — NumPy can operate on arrays of different shapes intelligently. (4) Rich functions — hundreds of mathematical, statistical, and linear algebra functions built in. The trade-off is that all elements in a NumPy array must be the same data type.

Q — What is broadcasting in NumPy?

Broadcasting is NumPy's ability to perform arithmetic operations between arrays of different shapes without explicitly copying data. When you multiply a (4,3) array by a (3,) array, NumPy

automatically stretches the smaller array to match the larger one and applies the operation element-wise. The simplest example is multiplying an array by a scalar: `prices * 1.18` adds 18% GST to every element at once. Broadcasting follows specific rules about shape compatibility, but for most practical cases it works intuitively.

Q — What does the shape attribute tell you about a NumPy array?

The shape attribute returns a tuple describing the dimensions of the array. A shape of (5,) means a 1D array with 5 elements. A shape of (3, 4) means a 2D array with 3 rows and 4 columns, like a table. A shape of (100, 28, 28) would describe 100 grayscale images each of 28 by 28 pixels. Understanding shape is critical in machine learning because almost every error about dimension mismatches is a shape problem. You can use `reshape()` to change the shape without changing the data, as long as the total number of elements stays the same.

Q — What is Boolean indexing in NumPy? Give an example.

Boolean indexing means using a True/False condition to filter elements from an array. When you write `scores[scores > 80]`, NumPy first evaluates `scores > 80` which produces an array of True and False values, then uses that mask to select only the elements where the condition is True. This is extremely efficient and is the primary way to filter data in NumPy and later in Pandas. For example, to get all students who passed: `passed = scores[scores >= 40]`. You can also combine conditions using `&` for AND and `|` for OR.

SECTION 10 Words to Know

Term	Definition
NumPy	Numerical Python. The core library for fast array operations in Python.
ndarray	NumPy's array object. n-dimensional array. The core data structure.
<code>import numpy as np</code>	The universal standard way to import NumPy. Always use this alias.
shape	Tuple describing array dimensions. (3,4) = 3 rows, 4 columns.
ndim	Number of dimensions. 1 = flat array, 2 = table, 3 = image.
dtype	Data type of elements: int64, float64, bool, etc.
<code>reshape()</code>	Change the shape of an array without changing its data.
Vectorisation	Applying an operation to a whole array at once, without writing a loop.
Broadcasting	Operating on arrays of different shapes. NumPy stretches the smaller one.
Boolean indexing	Filtering array elements using a True/False condition.
<code>axis=0</code>	Compute along rows (going downward). Gives one result per column.
<code>axis=1</code>	Compute along columns (going across). Gives one result per row.
<code>np.zeros()</code>	Create an array filled with zeros.
<code>np.arange()</code>	Create array of numbers with a specific step. Like <code>range()</code> for NumPy.

<code>np.linspace()</code>	Create array of evenly spaced values between two endpoints.
<code>np.random.seed()</code>	Fix the random starting point so results are reproducible.
<code>ufunc</code>	Universal function. A NumPy math function that works on entire arrays.
Normalisation	Scaling values to a 0-1 range: $(x - \min) / (\max - \min)$.

SECTION HW Homework — Submit Before Day 6

Create `day5_homework.py` and push to GitHub before midnight. First line of the file must be: `import numpy as np`

Exercise 1 — Array Creation Practice (Beginner)

Without using `np.array()` directly, create five arrays: (1) zeros of shape (3,5), (2) ones of shape (4,4), (3) integers from 10 to 50 with step 5 using `arange`, (4) 8 evenly spaced values between 0 and 1 using `linspace`, (5) a 3x3 identity matrix. Print each with its shape.

Exercise 2 — Student Marks Analyser (Beginner-Intermediate)

Create a 2D NumPy array with marks for 6 students across 4 subjects. Write code to: print the shape, find the highest and lowest mark in the entire table, compute each student's total and average (`axis=1`), compute each subject's average (`axis=0`), and use Boolean indexing to find all marks above 80.

Exercise 3 — Normalisation Function (Intermediate)

Write a function called `normalise(arr)` that takes a NumPy array and returns a new array where all values are scaled between 0 and 1. Test it on an array of prices: `[199, 499, 99, 999, 299, 149]`. Print the original and normalised arrays.

Formula: `normalised = (arr - arr.min()) / (arr.max() - arr.min())`

Exercise 4 — Broadcasting Practice (Intermediate)

Create a 4x3 array of random integers between 50 and 100 (use `np.random.randint`). Then: (1) add a bonus of `[3, 5, 2]` to each row using broadcasting, (2) subtract the column mean from each column to centre the data, (3) check how many elements in the final array are above 80.

Exercise 5 — Checkerboard Pattern (Intermediate-Stretch)

Create an 8x8 NumPy array of zeros. Then use slicing to set alternating squares to 1, creating a checkerboard pattern where the top-left corner is 0. Print the result. Hint: think about which row/column combinations should be 1.

Hint: `board[1::2, ::2] = 1` fills one set of squares. What fills the other set?

Stretch Challenge — Tier A students

Build a complete sales analysis tool using NumPy. Create a (7, 5) array representing 7 days of sales across 5 products (use random integers between 100 and 1000). Then: find the best-selling day (highest total), find the best-selling product (highest total across all days), find all days where total sales exceeded 3000, normalise the entire sales table to 0-1, and compute the percentage contribution of each product to total revenue.